

ABC Player - a walkthrough

ABC Player

A folk-tune player with a real parser and a real deadline — where every note begins on the sample it was written for, and the worst error measured is 0.021 milliseconds.

It reads ABC notation, the ASCII format that hundreds of thousands of traditional tunes are already written in, and plays it through either a chip-tune synthesiser or Apple's General MIDI synth. It also writes Standard MIDI Files, which is how the whole thing is proved correct by a program that was not written here.

Source	examples/abcplayer/ — ten files, ~4,900 lines
Reads	ABC: headers, accidentals, tuplets, repeats and endings, chords, ties, grace notes, multiple voices
Plays through	three chip voices, or Apple's DLS synthesiser
Writes	Standard MIDI Files
Time unit	ticks — 480 per quarter note, integer throughout
Timing error	1 sample, 0.021 ms, measured

These documents

Chapter	What it covers
1. Where it came from	ABC notation, a player written several times in several languages, and what the read-through turned up
2. The timing argument	Why sleeping until an event is due is the wrong design, and what replaces it
3. Reading ABC	A notation that is ambiguous exactly where real tunes live
4. Time as an integer	Ticks, repeats, ties, and the single conversion to samples

5. The audio thread	One schedule, two backends, and a callback that may not allocate
6. The window	A tune list, a voice editor, and Musical Typing
7. What to understand	The things that will surprise you, and the ones that will bite

The synthesiser it plays through has [its own walkthrough](#).

The shortest possible summary

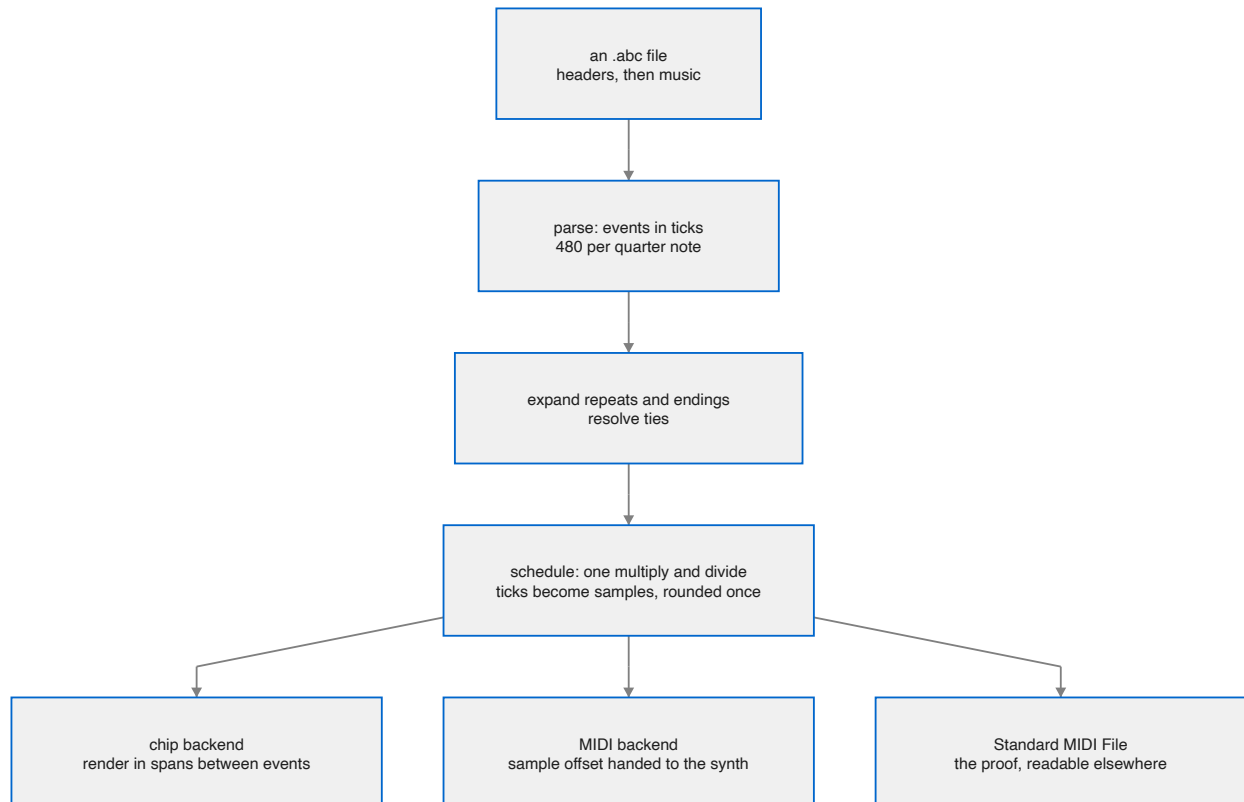
Most players schedule music by asking the operating system to wake a thread at a moment:

```
std::this_thread::sleep_until(start + std::chrono::duration<double>(when));  
sendMIDIEvent(event);
```

That is late by whatever the scheduler is busy with — a millisecond or two idle, tens under load. At 120 bpm a semiquaver is 125 ms, so a 5 ms error is **4% of a note**, and because it varies from note to note you hear it as looseness rather than as a tempo.

This player never sleeps. The tune is compiled to a list of *"at sample N, do this"*, and the audio callback applies each event at its exact offset **inside** the buffer. For the chip that means rendering in spans between events; for MIDI it means handing `MusicDeviceMIDIEvent` the sample offset it already accepts.

Everything upstream of that exists to make the sample number exact. Durations are integer ticks at 480 per quarter note — a resolution that divides evenly by everything ABC can ask for — and nothing rounds until one multiply and one divide at the very end.



1. Where it came from

Two histories meet in this program: a notation designed so that folk tunes could be sent by email, and a player that has been written several times over, in several languages, of which this is the latest.

ABC notation

ABC is a text format for music, devised by Chris Walshaw in the early 1990s for exactly the material this player targets: traditional and folk tunes, mostly single-line melodies, mostly in a handful of keys.

Its design constraint was plain-ASCII portability. A tune had to survive being typed into a newsgroup post or an email, so a note is a letter, its octave is a comma or an apostrophe, its length is a number, and a sharp is `^` because `#` was taken.

```
X:1
T:Sí Beag, Sí Mór
C:Turlough O'Carolan
M:3/4
L:1/8
K:D
D2|F4 A2|d4 f2|e4 d2|B4 A2|
```

Two properties of that design matter here. It is **terse** — good for humans, awkward for parsers. And it is **enormously widely used**: hundreds of thousands of tunes exist in it, which means a player is judged against real files written by people who were not thinking about your parser.

The awkwardness is not incidental complexity, and [chapter 3](#) is about it. Briefly:

- `(` opens a slur — unless a digit follows, when it opens a tuplet
- `[` opens a chord — unless it is `[K:`, when it is an inline field
- `|` is a bar line — unless a digit follows, when it also opens an ending
- an accidental holds **to the end of the bar**

Each of those ambiguities is resolved by looking at the *next* character, and all four occur in ordinary tunes.

One player, several languages

An ABC player is a good thing to port. It is small enough to finish, it has a genuinely awkward grammar, and the output is checkable — so it makes a decent way to get the measure of an unfamiliar language. This one has been through C++, Rust, Modula-2 and Smalltalk before arriving here, and the Mojo version took the C++ as its starting text simply because that was the handy one to read from.

The commit records what came out of that read-through — five behaviours the port had to change. They are worth keeping not as a verdict on any of the earlier versions but because of the property they share: **every one of them still produced music.**

None of them announces itself. A tune with the wrong key signature is still a tune; a note an octave low is still a note. Nothing crashes, nothing logs, and the output is plausible — which is exactly the class of fault that survives being tested by listening to it, and exactly why the [MIDI file output](#) earns its place.

1. Key signatures did nothing

`KeySignature("G")` set `sharps(0)` and never filled its `accidentals` map. Nothing anywhere assigned `.sharps`. And `applyKeySignature()` returned its argument unchanged, under a comment reading *"For now, just return the semitone unchanged"*.

So tunes played in C major whatever the `K:` field said. Folk material is mostly G, D and A, so that covers most of the corpus — and it still sounded like music, because a tune transposed into the wrong mode is still a tune.

2. Explicit accidentals were dropped

`isAccidental()` tested for `#` and `b`. Those are not ABC's accidental characters — ABC uses `^` and `-`.

It was also unreachable: the note branch was guarded by `isNote(*p)`, so `parseAccidental` only ran once the pointer was already on a letter. `^C` fell through to *"Ignoring unknown character"*.

3. First and second endings were ignored

`expandABCRepeats` duplicated the text between `|:` and `:|` with a regular expression. So `|: A |1 B :|2 C |` — which means *play A B, then A C* — came out as **A B A B**.

So a repeated strain ended on its first ending rather than its second. The `REP1` and `REP2` feature types were in the enum, waiting to be used.

4. Middle C was an octave low

`calculateMidiNote` computed `base_octave * 12 + semitone` with `base_octave = 4` for an uppercase note, giving 48, against a comment two lines below reading *"C4 = 60"*.

The model file now states the arithmetic and why the off-by-one octave is easy to land on:

*ABC's C is middle C, so the octave that letter sits in is 5 in the (octave * 12) convention — 5 * 12 + 0 = 60. Writing base_octave * 12 with base_octave 4 gives 48, which is a C, an octave low, and sounds plausible enough to survive review.*

The last clause is the point. An octave error does not sound broken; it sounds like a lower part.

5. Chord symbols corrupted the melody's timeline

`generateChordNotes` appended its tones to the **melody voice**, which advanced that voice's clock. So every `"Am7"` in the source pushed everything after it out of step.

Here chord symbols go to their own voice — `GCHORD_VOICE_BASE = 100` — so the melody's clock is untouched.

What the port took from that

The README puts the justification in one line:

Numbers 1 and 2 are why this port is not a rewrite for its own sake: a tune in K:D now plays in D.

Between them the first two cover most of what pitch means in folk music, which is what gives this port something to do beyond translating.

The more useful outcome is that two of the fixes are structural rather than corrective — the shapes that allowed the faults are gone, not just the faults:

A key signature is arithmetic, not a table.

The alteration a key applies to a letter follows from the circle of fifths, so `key_alter` computes it rather than storing seven numbers per key. The whole of Western key signatures is two orderings of the same seven letters.

A table can be half-filled and look fine. `key_alter` derives the answer from `sharp_order` — the sequence F C G D A E B — so there is no table to forget to populate. Bug 1 is not fixed here so much as made unrepresentable.

Repeats are expanded over events, not over text.

The expansion works on events rather than on text. By this point every note already knows its voice and its length, so replaying a section is copying a range and re-stamping the clock — and a repeat that crosses a line break, or sits inside one voice of several, needs no special handling at all.

The regex approach that produced bug 3 operates on characters, which know nothing about voices or line breaks. Working on events makes the hard cases disappear rather than requiring extra code.

What this one is testing

Each language this player has been written in gets tested on the same two things, and the README names them:

It was worth doing for two reasons that have nothing to do with Mojo being new: a complicated parser is a real test of a language, and a music player is a real test of timing.

The parser half is [chapter 3](#). The timing half is where this version departs from its predecessors rather than merely translating them — and it is [chapter 2](#).

2. The timing argument

This is the chapter the player exists for, and it is the same shape of argument the Othello example makes: a widely-used design is measured, found wanting, and replaced by one that is measured too.

The design almost everybody uses

An event scheduler usually looks like this, and it is what the earlier versions of this player did:

```
std::this_thread::sleep_until(start + std::chrono::duration<double>(when));
sendMIDIEvent(event);
```

Compute when the note is due, sleep until then, send it. It is the obvious design, it is what almost every player does, and it has a ceiling you can hear.

It asks the operating system to wake a thread at a moment. The kernel will oblige when it next gets round to it — late by whatever the scheduler is busy with. A millisecond or two on an idle machine; tens of milliseconds under load, when another process is compiling or a browser is laying out a page.

The source does the arithmetic:

At 120bpm a semiquaver is 125 ms, so a 5 ms error is 4% of a note — audible as looseness, and worse, it varies.

The variation is the part that matters. A **constant** 5 ms delay is inaudible; it is a latency, and every note has it. An error that changes from note to note is *jitter*, and jitter is exactly what the ear is built to detect. It does not sound like a slow player. It sounds like a bad one.

There is a second problem stacked on the first. Sleeping schedules the *send*, but the synthesiser only produces sound when the audio callback next runs. So even a perfectly-woken thread has its event quantised to the next buffer boundary — at 512 frames and 48 kHz, another 10.7 ms of slop.

What replaces it

The tune is compiled ahead of time into a list of **"at sample N, do this"**, and the audio callback applies each event at its exact offset *inside* the buffer it is currently filling.

```
@fieldwise_init
struct Step(implicitlyCopyable, Copyable, Movable):
    """One thing to do, at one sample."""

    var sample: Int
    var kind: Int
    var voice: Int
    var midi: Int
    var velocity: Int
```

No thread is woken. Nothing sleeps. The only clock is the one the audio hardware is already running, and a sample index is an exact integer.

The header of `schedule.mojo` states the claim and where it can fail:

This is where the timing claim is either true or not. A tick is an exact integer; a sample is an exact integer; the conversion between them is one multiply and one divide in 64-bit, rounded once. Nothing accumulates.

Scheduling by sample index instead means an event lands on the sample it was written for, and the only jitter left is the buffer boundary, which the callback already knows about and can subtract exactly.

That last clause is the whole trick. The buffer boundary does not disappear — it is just *known*, so it can be subtracted rather than suffered.

How each backend honours it

The two backends solve the same problem differently, and neither has any timing code of its own.

The chip renders the buffer in **spans between events**:

If a note begins 137 samples into a 512-sample buffer, the first 137 samples are rendered, the note is started, and the remaining 375 are rendered after — so the note begins on sample 137 and not at the buffer boundary. That is the whole difference between sample-accurate and buffer-accurate timing, and it costs one loop.

MIDI does not need the loop, because the API already takes the offset:

MusicDeviceMIDIEvent takes an offset in samples from the start of the buffer being rendered, so an event 137 samples in is applied 137 samples in — the same accuracy the chip backend gets by rendering in spans, and the reason this player does not need a scheduling thread at all.

That parameter has been in CoreAudio all along. Most software passes zero.

The measurement

The claim is testable, and the test is designed to be hostile — a buffer size chosen so that note onsets *cannot* land on buffer boundaries:

Measured, playing quarter notes at 120bpm through 512-frame buffers — a size that deliberately does not divide the note length, so every onset falls mid-buffer.

```
scheduled  0    48000    96000    144000
measured   0    48001    96001    144000
worst error 1 sample = 0.021 ms
```


0.021 ms, against 5 ms or worse for the sleeping design — better by more than two orders of magnitude. And the source declines to claim even that much:

The one-sample errors are the onset detector's threshold, not the scheduler.

Two of the four onsets are off by one sample, and the honest reading is that the *measuring apparatus* has a one-sample threshold. The scheduler may well be exact. The report does not claim it, because the experiment cannot distinguish the two.

Why the design has a second payoff

The most useful consequence is not accuracy. It is that **there is no timing code in either backend**:

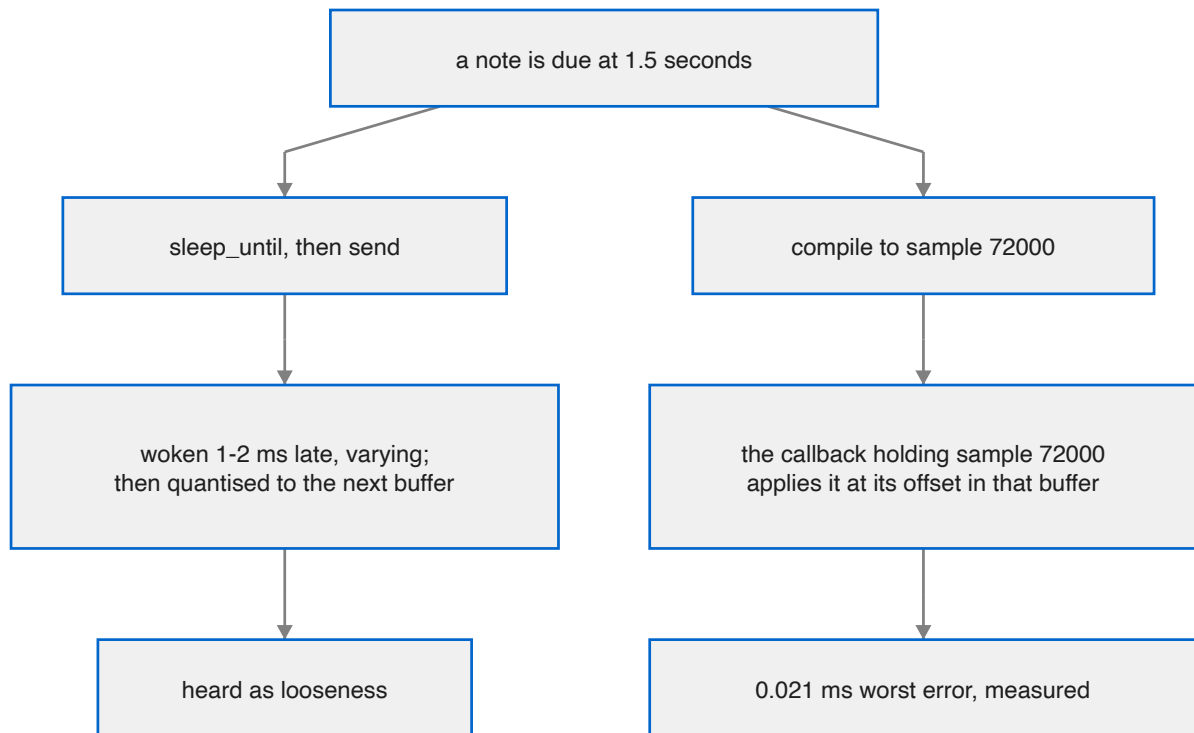
Both backends share one render callback and one schedule. That is the point of the design: the tune is turned into a list of "at sample N, do this", and the only thing a backend decides is what "this" means. Nothing about timing is duplicated, so nothing about timing can differ between them.

A sleeping scheduler needs a thread, a wake-up calculation, a way to stop it, and a way to keep it in step with a second backend that has its own latency. All of that is absent here. Adding a third backend would mean writing what "note on" means and nothing else.

And it removes an entire class of concurrency bug. There is no scheduling thread to race with the audio thread, no queue between them, and no lock. The schedule is written once, before the audio unit starts, and is read-only thereafter.

The comparison

	sleep-until	scheduled by sample
clock	the OS scheduler	the audio hardware
error, idle	1–2 ms, varying	0.021 ms measured
error, under load	tens of ms	unchanged
quantisation	to the next buffer	none
threads	one extra, plus a queue	none
timing code per backend	one implementation each	none



3. Reading ABC

The parser is three files: `parse.mojo` splits a file into headers and music, `music.mojo` reads the music, and `model.mojo` holds what they produce. It is about 1,500 lines, and the header of `music.mojo` explains why it needs to be:

This is the part with the sharp edges. The notation is terse, ambiguous in places, and every real tune uses the ambiguous parts.

The one rule that makes ABC parseable

```
# ABC's header ends at the K: field, which is the one rule that makes the
# format parseable at all -- everything before it configures the tune, and
# everything after it is music, except that field lines are still allowed in
# the body and most real tunes use them for voice switches.
```

K: is the boundary. Before it, every line is a field; after it, a line is music **unless** it looks like a field — one letter, a colon, a value.

That exception is not a nicety. Multi-voice tunes switch parts with **V:** lines in the middle of the music, and **L:** or **K:** can change mid-tune. So the parser cannot simply stop looking for fields; it has to keep the test running and let real tunes use it.

Four ambiguities, each resolved by one character of lookahead

written	usually	unless
(	a slur	a digit follows — then a tuplet
[	a chord	it is [K: etc. — then an inline field
	a bar line	a digit follows — then also an ending
^F	this F is sharp	every F for the rest of the bar is

The first three are ordinary lookahead. The fourth is different in kind, and the source flags it:

That last rule is the one worth being careful about. It is what makes ABC readable to a musician, and a parser that skips it plays wrong notes in roughly every tune that has an accidental in it.

This is the convention of printed music: write the accidental once, and it governs that letter until the bar line. A parser that treats `^F` as "this note is sharp" gets the second F in `^F A F` wrong — and gets it wrong in almost every tune that has an accidental at all, because writing it twice is what notation exists to avoid.

The accidental state, in one integer

The bar's accidentals live in a single `Int`, seven nibbles wide:

```
var bar_acc: Int          # seven 4-bit fields, one per letter

fn acc_of(self, letter: Int) -> Int:
  """The accidental in force for a letter this bar, or NO_ACCIDENTAL."""
  let nib = (self.bar_acc >> (letter * 4)) & 0xF
  return NO_ACCIDENTAL if nib == 0 else nib - 8

fn set_acc(mut self, letter: Int, alter: Int):
  # Biased by 8 so that zero can mean "nothing set": a natural sign is
  # an accidental too, and storing it as 0 would erase it.
  let nib = (alter + 8) & 0xF
  self.bar_acc = (self.bar_acc & ~(0xF << (letter * 4))) | (
    nib << (letter * 4)
  )
```

The bias by 8 is the detail worth noticing. A natural sign (`=F`) is an alteration of **zero**, and it is a real instruction — it cancels a sharp earlier in the bar. Storing alterations raw would make "natural" and "nothing set" the same value, and `=F` would silently do nothing.

Clearing the bar is then one assignment:

```
fn clear_bar(mut self):
  self.bar_acc = 0
```

Which is called at every bar line, and also after `K:` — because a key change resets what the letters mean.

A key signature is arithmetic

Seven letters, two orderings, no tables:

```
fn sharp_order(k: Int) -> Int:
  """Sharps arrive F C G D A E B; flats take the same list backwards."""

fn key_alter(sharps: Int, letter: Int) -> Int:
  """What a key signature does to one letter: +1, 0 or -1.

  Derived rather than stored. With `sharps` sharps, the first `sharps`
  letters of F C G D A E B are raised; with flats, the first `-sharps` of
  that list read backwards are lowered.
  """
```

Sharps always arrive in the order F C G D A E B; flats arrive in exactly that list reversed. Two sharps means F and C are raised; that *is* the definition of D major. There is nothing to look up.

`key_sharps_for` then maps a `K:` field onto a position on the circle of fifths, and handles the modes — which are not decoration:

a tune marked `K:Ador` is in G major's signature, and reading it as A major puts three accidentals in the wrong place for the whole tune.

`Ador` is A Dorian: A as the tonic, but G major's key signature. Folk material uses modes constantly — Dorian and Mixolydian especially — so a player that treats the letter as a major key is wrong for a large fraction of the corpus. The implementation is a subtraction:

```
elif rest.startswith("dor"):
    fifths -= 2
```

Dorian is the second mode, so its signature sits two fifths below the major of the same letter. Mixolydian is one below, minor and Aeolian three, Phrygian four, Locrian five, Lydian one above. Six lines for the whole modal system.

Durations, and why nothing is a float

```
def read_duration(line: String, start: Int, unit_ticks: Int) -> List[Int]:
    """The length that follows a note, rest or chord. Returns [ticks, next].

    ABC writes a multiplier, a divisor after `/`, or both: `A2`, `A/2`, `A/`,
    `A//`, `A3/2`. A bare `` halves, and each extra `/` halves again.
    """
```

`A3/2` is a dotted note. `A//` is a sixteenth of the unit length. All of it is a rational multiple of `L:`, and all of it stays exact because the unit is 480 ticks per quarter — see [chapter 4](#).

State the parser has to carry

```
struct MusicCtx(Copyable, Movable):
    var voice: Int
    var bar_acc: Int           # seven 4-bit fields, one per letter
    var tuplet_left: Int       # notes still inside a tuplet
    var tuplet_num: Int
    var tuplet_den: Int
    var broken: Int           # >0 after '>', <0 after '<', magnitude = count
    var in_grace: Bool
    var last_note: Int        # index in tune.events of the last note emitted
    var gchord_root: Int      # the chord symbol in force, or -1
    var gchord_kind: Int
```

Everything here exists because ABC lets it change mid-tune. `tuplet_left` counts down over the *following* notes, so `(3` alters the three notes after it. `broken` records a `>` or `<` and applies to the pair

it sits between. `last_note` is needed because a tie and a broken rhythm both reach backwards to modify a note already emitted.

The `Voice` struct carries the same idea at a larger scale:

ABC lets any of this change mid-tune through an inline field, so it is state rather than configuration.

The subtle one: defaults for a voice that has not appeared yet

```
# Header defaults. A voice can be mentioned for the first time in the
# music, long after L: and K: were read, so the tune has to remember what
# a new voice should start life with -- otherwise a tune whose header says
# L:1/4 plays in eighths and nothing in the parse looks wrong.
var default_unit_num: Int
var default_unit_den: Int
...
```

`V:2` may appear forty lines below the `L:1/4` that should govern it. If the new voice is created with library defaults instead of the tune's, it plays at the wrong note length — and *nothing in the parse looks wrong*, because every individual step succeeded. The tune simply has one part in the wrong tempo.

Encoding traps, met and commented

Two byte-level details cost time and are recorded where they happened.

A trailing carriage return:

```
# A trailing carriage return is invisible and breaks every comparison
# that follows it, so it goes first. The trimmed copy is a new String:
# assigning a slice of a String back over itself aliases the buffer it
# is reading from.
```

ABC files come from everywhere, so CRLF is common. `"K:D\r"` is not `"K:D"`, and the difference is invisible in every error message. The second sentence is a separate hazard in this dialect: slicing a `String` back over itself aliases the buffer being read.

Rebuilding a string byte by byte:

```
# Sliced, not rebuilt character by character: `chr(byte)` turns
# each byte of a multi-byte character into its own codepoint, so
# a title with an accent in it comes out as mojibake.
let body = String(line[byte = 2 : len(b)])
```

`T:Sí Beag, Sí Mór` is the ordinary case, not an exotic one. Iterating bytes and calling `chr` on each turns every UTF-8 continuation byte into its own character.

What is deliberately not supported

Ignored deliberately, because a chip cannot hear them: slurs, decorations, lyrics alignment, and clefs.

A slur is a phrasing mark for a human player; a chip has no legato to apply. **w:** lyric lines are consumed and dropped. The decision is recorded rather than left as an omission, which is the difference between a limitation and a bug.

4. Time as an integer

Everything in [chapter 2](#) rests on the sample number being *exactly* right. This chapter is how it gets that way: one integer unit from the parser to the last possible moment, and a single rounded conversion at the end.

480 ticks to the quarter note

```
comptime TICKS_PER_QUARTER = 480
comptime TICKS_PER_WHOLE = TICKS_PER_QUARTER * 4
```

Durations are counted in ticks at 480 per quarter note — 1920 per whole note — and never in seconds or doubles. That number is the standard MIDI resolution and it divides exactly by everything ABC can ask for: a 1/64 note is 30 ticks, a triplet eighth is 160, a dotted quarter is 720.

Check the awkward cases:

written	ticks	exact?
whole note	1920	yes
1/64 note	30	yes
triplet eighth ($240 \times 2/3$)	160	yes
dotted quarter ($480 \times 3/2$)	720	yes
quintuplet quarter ($480 \times 4/5$)	384	yes
1/128 note	15	yes

$480 = 2^5 \times 3 \times 5$, which is what makes it divisible by 3 (triplets), 5 (quintuplets), 2 repeatedly (dots and short notes), and 7 only approximately — septuplets are the one case that does not land, and ABC tunes essentially never use them.

The alternative is stated as a failure mode rather than a preference:

Floating-point timestamps drift by a fraction of a tick per note, which is inaudible until a few hundred notes have gone by and the voices are visibly apart.

That is the characteristic shape of the bug: a tune sounds fine and its parts drift apart over a minute. With integers there is no accumulation, because there is no rounding to accumulate.

The single conversion

```
fn tick_to_sample(tick: Int, bpm: Int, per_beat: Int, sample_rate: Int) -> Int:
    """Exact, and rounded once at the end rather than accumulated.
```



```

sample = tick * (60 / bpm) * sample_rate / ticks_per_beat, done as one
integer expression. The numerator is at most a few times 10^12 for any
real tune, which is comfortably inside 64 bits.
"""
let denom = bpm * per_beat
if denom <= 0:
    return 0
return (tick * sample_rate * 60 + denom // 2) // denom

```

One expression. Three multiplies and one divide, on a value computed from the event's **absolute** tick — not from the previous event's sample, which is what would accumulate.

The `+ denom // 2` is round-to-nearest rather than truncation, so the error is at most half a sample rather than up to a whole one — which is the difference between the measured worst case being one sample and being two.

The range note is doing real work too: `tick * sample_rate * 60` for a ten-minute tune at 48 kHz is around 10^{12} , against a signed 64-bit ceiling of 9.2×10^{18} . Six orders of magnitude of headroom, checked rather than assumed.

Repeats, expanded over events

```

# `|`: A |1 B :|2 C |` plays A B A C. That is the whole point of the notation
# and it is the case naive expanders get wrong: duplicating the text between
# `|`:` and `:|` gives A B A B, so the tune ends on the wrong phrase every
# time. Endings are not decoration; they are how a repeated strain gets out
# of itself.

```

The mechanism is one walk with one rule:

```

elif ev.kind == EV_BAR and ev.aux == BAR_REPEAT_END:
    order.append(own[i])
    # The replay stops where the first ending begins, so the
    # second pass falls through into the second ending.
    let stop = ending1 if ending1 >= 0 else i
    for k in range(repeat_start, stop):
        order.append(own[k])

```

At `:|`, replay from the repeat start — but stop at the first ending if there was one. The second pass therefore runs off the end of the replay and continues into whatever follows, which is the second ending. No special case for `|2` at all; it is simply the code that comes next.

And the reason this is done on events rather than text:

By this point every note already knows its voice and its length, so replaying a section is copying a range and re-stamping the clock — and a repeat that crosses a line break, or sits inside one voice of several, needs no special handling at all.

A regex over text cannot see voices or line breaks. Working on events makes those cases vanish rather than requiring code.

Re-stamping the clock, and the chord trap

Copying events is only half the job — the copies need new tick numbers. The rule is that **only things which occupy time advance the clock**:

```
if (ev.flags & F_CHORD) != 0 or (ev.flags & F_GRACE) != 0:
    # A chord's later members sound with the note that opened the
    # group, which is the tick *before* it advanced the clock --
    # using the current tick instead spreads a chord out into an
    # arpeggio, one member per beat, which is audibly wrong and
    # looks like a parser bug rather than a re-timing one.
    ev.tick = group_tick
    expanded.append(ev)
    continue
group_tick = tick
ev.tick = tick
expanded.append(ev)
tick += ev.duration
```

Three kinds of event, three behaviours: a bar line is a marker and takes the current tick; a chord member or grace note takes `group_tick`, the tick of the note that opened its group; an ordinary note takes the current tick and then advances it.

The comment records the symptom as well as the cause, and the symptom is the useful part: get it wrong and [CEG] comes out as C, E, G one beat apart. That looks exactly like a chord-parsing bug — and the parser is fine. The fault is in re-timing, several stages downstream.

Ties

```
def resolve_ties(mut tune: Tune):
    """Join a tied note to the note it is tied to.

    A tie means one sound, not two: the second note is not struck. Extending
    the first and dropping the second is what makes `C-C` a half note and not
    two quarters with a seam.
    """
```

Two details in the implementation are worth extracting.

The match has to be exact in three ways — same voice, same pitch, and starting *precisely* where the first ends:

```
if tune.events[j].tick < end:
    continue
if tune.events[j].tick > end:
    break
```

Past the end, stop looking. A tie that does not find its partner leaves the first note at its written length, which is the right failure.

The second note is silenced, not deleted:

```
# Silenced rather than removed: removing it would move every
# index after it, and the outer loop is holding one.
tune.events[j].velocity = 0
```

A classic. Deleting from a list you are iterating invalidates the position you are standing on. Setting velocity to zero is then honoured downstream by one line in the schedule builder:

```
if ev.velocity <= 0:
    continue                # swallowed by a tie
```

Sorting, and the tie-break that saves a note

```
def sort_steps(mut steps: List[Step]):
    """Merge sort by sample, with note-off before note-on at the same instant.

    The tie-break matters: when one note ends exactly as another begins on the
    same voice, releasing first leaves the voice free for the new note. The
    other order steals a voice that is about to be freed and drops a note.
    """
```

```
fn step_before(a: Step, b: Step) -> Bool:
    if a.sample != b.sample:
        return a.sample < b.sample
    return a.kind > b.kind          # NOTE_OFF (1) sorts before NOTE_ON (0)
```

With only three chip voices, voice allocation is tight. Two events at the same sample — one note ending, another beginning — must be applied in the order that frees a voice before asking for one. The other order takes a voice from a sounding note while a voice was about to become free, and the tune loses a note it did not have to.

Note also that the sort is a **bottom-up merge sort**, which is stable — so events written in a particular order at the same sample stay in that order beyond the explicit tie-break.

The release gap

```
# Lift the note a little early so repeated pitches are re-struck
# rather than running together. A twentieth of the note, capped, is
# about what a player does without thinking about it.
var gap = ev.duration // 20
let max_gap = per_beat // 16
```

```
if gap > max_gap:
    gap = max_gap
```

Two notes of the same pitch back to back, released exactly as the next begins, sound like one long note — the ear needs a gap to hear a re-articulation. Five percent of the note, capped at a sixteenth of a beat so long notes do not get a long silence, is a musical judgement expressed in two lines.

The MIDI file as proof

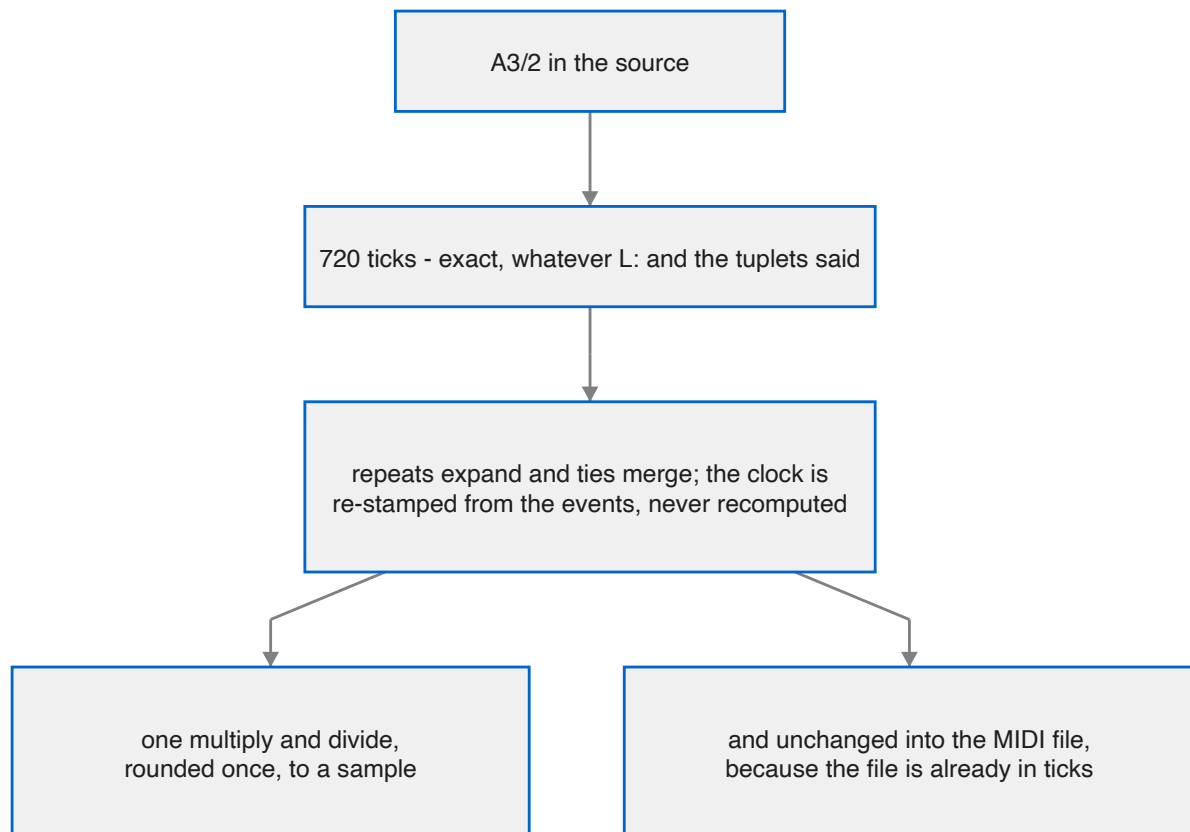
```
# This is the unglamorous half, and it is also the half that proves the model
# is right. A MIDI file is a public format with other readers: if the tune
# opens in a notation program with the right pitches, the right lengths and
# the bar lines in the right places, then the parser and the tick arithmetic
# are correct, and no amount of listening to a synthesiser could establish
# the same thing.
```

This is the strongest verification idea in the program. Listening tests your ears. A MIDI file opened in notation software tests your model **against a reader that was not written here** — and it displays the result as notation, so a bar that does not add up is visible rather than audible.

And the tick resolution makes the export trivial:

The tick resolution is already MIDI's own — 480 per quarter note — so durations transfer with no conversion and no rounding. That is not a coincidence; it is why 480 was chosen in the model.

The design decision at the top of this chapter pays off at the bottom of it: no conversion means no rounding means the file says exactly what the model says.



5. The audio thread

CoreAudio calls a render callback on a real-time thread it owns, every 10.7 ms at 512 frames and 48 kHz. That callback must fill the buffer before the speaker runs dry, and it **may not allocate, may not take a lock, and may not raise**. Miss the deadline and you hear it.

This chapter is what the player does inside that constraint.

The callback is an ordinary Mojo function

```
comptime AURenderCallback = fn(P, P, P, UInt32, UInt32, P, /) -> Int32

fn render(ref_con: P, action_flags: P, timestamp: P,
          bus: UInt32, frames: UInt32, io_data: P) -> Int32:
    """Fill one buffer. Runs on CoreAudio's real-time thread."""
```

`fn` in this dialect is a **foreign-callable, non-raising, C-ABI function**. So `render` is an `AURenderCallback` — its address goes straight into `AudioUnitSetProperty` with no shim, no C file, and no Objective-C.

That the language's non-raising function kind is also its C-callable one is not a coincidence here; it is exactly the contract CoreAudio requires, enforced by the type rather than by discipline. A `def` may raise, and the compiler will not let you install one.

The buffer list is unpacked by hand, because it is a C structure with a variable-length tail:

```
"""The buffer list is two non-interleaved float channels: an AudioBuffer is
{UInt32 channels; UInt32 bytes; void* data} and the list's array starts at
offset 8, so channel i's data pointer sits at 16 + i*16."""
```

One callback, two backends

```
if get(st, PLAYER_BASE + UI_BACKEND) == BACKEND_MIDI:
    render_midi(st, action_flags, timestamp, frames, io_data, n)
else:
    render_scheduled(st, left, n)
```

One branch, at the top, and after it the two backends share nothing except the schedule they read. Neither contains a line of timing code.

The chip backend: rendering in spans

This is the loop that makes the timing claim true.

```
var filled = 0
```

```

while filled < frames:
    var cursor = get(st, PLAYER_BASE + SC_CURSOR)
    let now = get(st, PLAYER_BASE + SC_SAMPLE)

    # Everything due at this exact sample happens before another sample
    # is rendered.
    while cursor < count:
        let at = cursor * STEP_SLOTS
        if sched[unsafe_offset=at] > now:
            break
        ...apply the event...
        cursor += 1
    put(st, PLAYER_BASE + SC_CURSOR, cursor)

    # Render as far as the next event, or to the end of the buffer.
    var span = frames - filled
    if cursor < count:
        let next = sched[unsafe_offset=cursor * STEP_SLOTS] - now
        if next < span:
            span = next
    if span < 1:
        span = 1

    chip_render(st, dest + filled, span, silent_tick)
    filled += span
    put(st, PLAYER_BASE + SC_SAMPLE, now + span)

```

Read it as: *apply everything due now; render up to the next thing that is due; repeat until the buffer is full.*

A buffer with no events is one `chip_render` call of 512 samples. A buffer with a note at offset 137 is two calls, of 137 and 375. The cost is a loop iteration per event, which is nothing, and the benefit is that a note begins on its own sample.

The `if span < 1: span = 1` guard is a liveness fix rather than a correctness one — two events at the same sample would otherwise give a zero-length render and the loop would not advance.

The MIDI backend: an offset the API already takes

```

_ = external_call["MusicDeviceMIDIEvent", Int32](
    synth, UInt32(0x90 | channel), UInt32(note),
    UInt32(velocity), UInt32(offset),
)

```

That last argument is `inOffsetSampleFrame`. It has been in CoreAudio for decades and most software passes zero. Passing the real offset is the entire MIDI-side implementation of sample-accurate timing.

The channel mapping carries one piece of MIDI folklore:

```

if channel >= 9:
    channel += 1

```

Channel 10 (index 9) is General MIDI's percussion channel — notes sent there are drums, not pitches. Voices skip over it, or a third-voice harmony line would come out as a cymbal.

No allocation, and how that is guaranteed

The schedule is a `List[Step]` while it is being built, which allocates. It is flattened into plain memory before the audio unit starts:

```
def flatten_schedule(steps: List[Step], mut st: P) -> Int:
    """Copy the schedule into plain memory the audio thread can walk."""
    let addr = Int(external_call["calloc", P](Int(n * STEP_SLOTS + 8), Int(8)))
```

Everything here runs on the audio thread, so the schedule is a plain block of memory rather than a List: allocated once, before the unit starts, and never resized.

Five `Int`s per step, at a fixed stride. The audio thread walks it with a cursor and never writes to it.

The same rule shapes two other functions.

`midi_to_hz` **clamps its input**, and the reason is severity rather than correctness:

```
"""Clamped, because this runs on the audio thread and the octave loop below
is a loop: a nonsense note would be a hang, not a wrong pitch."""
```

A wrong pitch is a wrong note. A hang on the audio thread is silence for the rest of the session, and it is the harder failure to diagnose because nothing crashed.

And there is a do-nothing function whose docstring explains why it exists:

```
fn silent_tick(st: P) -> None:
    """The chip's own player routine, doing nothing.

    The schedule drives the notes here, so the 50 Hz routine has no work --
    but the chip still calls it, and a null function pointer would not do.
    """
    pass
```

The chip expects a 50 Hz player routine — see the [synth walkthrough](#). This player drives notes from the schedule instead, so the routine has nothing to do, and the honest implementation is an empty function rather than a null pointer and a branch.

Three voices, and what gives when a fourth note arrives

```
"""Three voices and a tune that may want more, so something has to give when
they are all busy. A voice whose envelope has finished is free; failing
that the oldest sounding note is taken, because it is the one furthest
through its decay and the least missed."""
```


Three tiers, in order:

1. a voice holding no note at all
2. a voice whose envelope has reached `ENV_IDLE` — still assigned, but silent
3. otherwise, the **oldest** sounding note

Tier 2 is the one that matters musically. A released note holds its voice while it decays, and once the envelope is finished that voice is audibly free even though the bookkeeping still points at it. Checking the envelope recovers a voice that would otherwise be wasted.

Tier 3's justification is the honest one: whichever note has been sounding longest is furthest through its decay, so stealing it is the least audible choice available.

Register changes are events too

An `[I:chip ...]` directive in the tune becomes a `Step` like any other:

```
fn apply_chip(st: P, voice: Int, param: Int, value: Int):
  """One register change, on the audio thread.

  No allocation and nothing that can raise: this is the same contract the
  note events keep, because it runs from the same place they do.
  """
```

A directive is not special machinery. It becomes a `Step` at a sample position exactly like a note-on.

So a filter sweep written into a tune lands mid-phrase, on the sample it was written for, with the same accuracy a note gets — because it is travelling through the same code.

One consequence needed extra state:

```
fn record_adsr(st: P, voice: Int, a: Int, d: Int, sus: Int, r: Int):
  """Remember the four nibbles a set_adsr was given.

  set_adsr turns them into 16.16 increments through a period-stretching
  ladder, and that is not invertible -- so changing only the decay later
  means keeping the other three somewhere.
  """
```

`set_adsr` takes four nibbles and computes increments. The transform loses information, so changing one parameter later requires the original four — kept in the player region where the audio thread reads them without a lock.

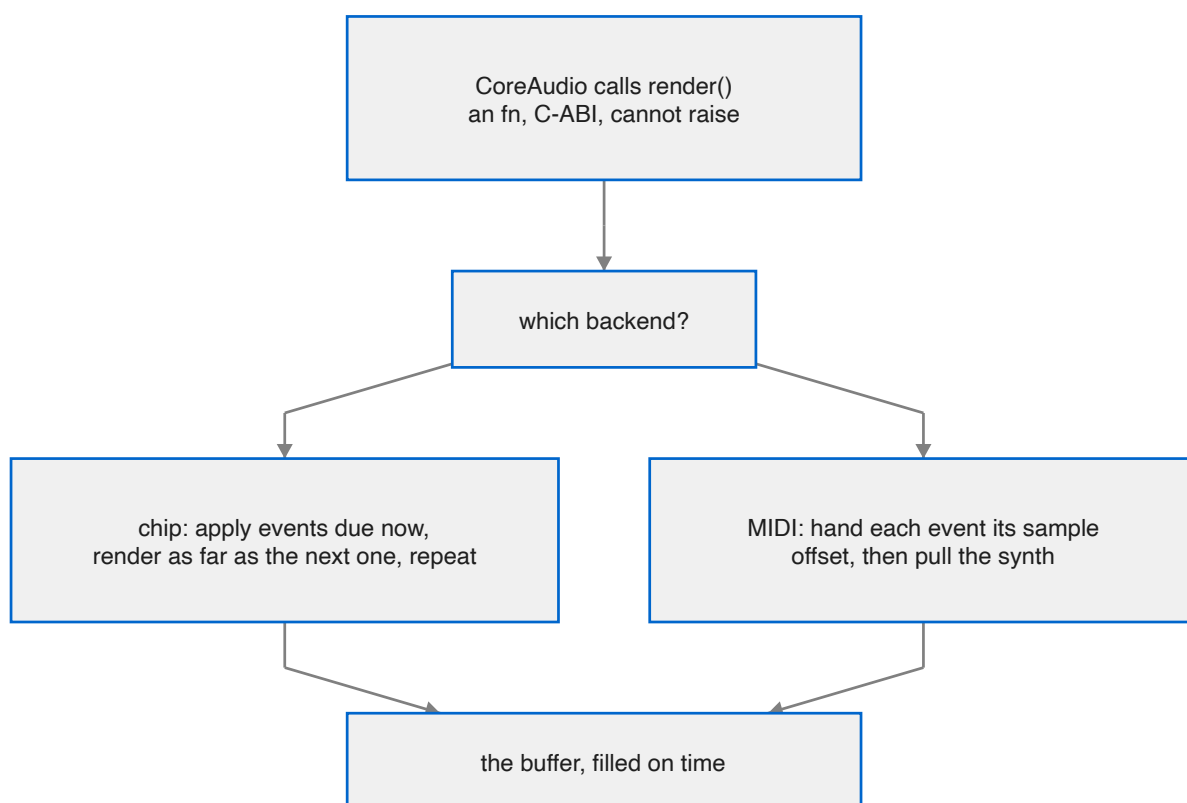
Why there is no lock anywhere

The window can move a slider while the tune plays. That is two threads touching the same registers, and there is no mutex:

Nothing here touches the audio thread's schedule. Live notes are register writes — set a frequency, raise a gate — and the render callback reads those registers on its own clock. A torn read costs one frame of one oscillator, where a lock would cost a click in the speaker.

The reasoning is specific rather than cavalier. The shared data is individual integer registers; the worst outcome of a torn read is that one oscillator uses a stale value for one 50 Hz frame, which is inaudible. A lock, meanwhile, can make the audio thread wait on the UI thread — and a late buffer is a click, which is very audible indeed.

Blocking is the more dangerous failure on a real-time thread, and that is what makes the trade correct here rather than merely convenient.



6. The window

`ui.mojo` is 1,018 lines and draws everything itself:

There is no `NSButton` and no `NSSlider` — the panel is a few dozen rectangles and a hit test, which is less code than wiring that many controls to target/action, and it keeps the whole interface in one place you can read.

That is a defensible claim for this interface specifically. Roughly forty controls, each of which sets one integer register, would be forty outlets and forty actions. Drawn directly, the whole panel is a layout function and a hit test — and adding a register is one entry in one table.

Three things live in it

A tune list. Whatever is in `tunes/` at startup, plus anything **Add...** brings in through a file panel.

A voice editor showing all three voices at once. The reason is musical rather than aesthetic:

There are only three, and a patch is the relationship between them — which voice carries the melody, which one is the noise channel — so none of them is hidden behind a tab.

Every register the chip actually has is on screen. Two details in that layout are worth extracting:

the four waveforms (they AND together, which is what the hardware did, so they are toggles and not a radio group)

The interface tells you the truth about the hardware. On the real chip selecting two waveforms ANDs their outputs — which is where most of the memorable timbres come from — so presenting them as mutually exclusive would both remove capability and teach the wrong model.

Cutoff, resonance and level sit in their own block underneath, labelled, because they belong to the one filter all three voices share — a control that looks per-voice and is not would be worse than no label at all.

There is one filter, not three. Putting cutoff in each voice's column would imply three independent filters, and the user would spend a while wondering why voice 2's cutoff moves voice 1.

A playable keyboard, using Logic Pro's and GarageBand's *Musical Typing* layout:

W E T Y U O P	the black keys, in their piano positions
A S D F G H J K L ;	the white keys, from C

the mapping a Mac musician already has in their fingers

R and **I** are gaps because a piano has no black key between E and F, or between B and C — the layout is a picture of a keyboard, so the gaps have to be there.

Key	
Z X	octave down / up
C V	level down / up
Tab	sustain
Space	play / pause
Q Esc	quit

Z X C V are free for that job precisely because they are not note keys, which is why Logic chose them and why this does too.

Following an existing convention rather than inventing one gets the layout *and* its reasoning for free.

Velocity is deliberately absent

The chip has no per-note velocity — neither did the 6581 — so C and V move the master level, which is the register that exists.

This is the most instructive decision in the interface. A velocity control would be easy to add and would do nothing, because there is no register behind it. Rather than fake one, the keys move the register that is real.

The same honesty governs polyphony:

Three voices means three notes. A fourth steals the oldest, which is what a three-oscillator chip has to do.

Live playing does not touch the schedule

```
# Nothing here touches the audio thread's schedule. Live notes are register
# writes -- set a frequency, raise a gate -- and the render callback reads
# those registers on its own clock. A torn read costs one frame of one
# oscillator, where a lock would cost a click in the speaker.
```

Playing a key does not enqueue anything. It writes a frequency and raises a gate, and the render callback picks that up whenever it next looks. The [audio chapter](#) covers why there is no lock.

The panel reads back the same way:

The panel reads the chip's registers rather than its own copy of them, so you can watch a tune move the sliders.

One source of truth. A tune containing `[I:chip cutoff=450]` moves the cutoff slider, because the slider is drawn from the register the directive wrote.

Changing the sound mid-tune

The directive lives in ABC's `I:` field, and the choice of field is argued:

ABC's I: field is specified as instructions to the software, which is exactly what a chip register is, so the directive goes there rather than inventing syntax nobody else would recognise.

```
[I:chip v=1 wave=pulse pw=1100 a=0 d=6 s=9 r=3 cutoff=1200 res=7 vol=11]ACEA cAec
[I:chip v=1 pw=350 cutoff=450]aged cAEC
```

Because it is written inline, it applies at the point in the music where it appears — not at a bar line, not at the start of a phrase.

And unknown keys are ignored rather than refused:

so a tune using a setting this build does not have still plays its notes.

Forward compatibility in one sentence. A tune written for a later version with more registers still plays; it just sounds plainer.

The demonstration tune

`tunes/galixigans.abc` exists to show the directive doing something a static patch could not:

a dive where the pulse width narrows and the filter closes as they come at you, a four-step power-up with the cutoff opening on every bar, and a fanfare. It is in A minor throughout the defence and ends in A major. The picardy third is the joke — it is a triumphant ending, and the triumph is not ours.

A **picardy third** is the old device of ending a minor piece on a major chord — a lift at the last moment. Here the aliens win and the music is triumphant anyway.

Two things that make the window testable

`ABC_SHOT=<path>` draws one frame to a PNG and exits, so the window can be checked without a person at the screen.

The same idea as Fluid's `FLUID_AUTOSHOT`: a GUI you can only check by looking at it is a GUI nobody checks. One environment variable makes the interface part of a scriptable run.

And the fonts are made once, which is a fix rather than an optimisation — the chip example's README records what happens otherwise:

Asking for a font on every string, thirty times a second, eventually returned nil, and a nil value in an attributes dictionary raises — surfacing as a trap deep inside AppKit with a stack that mentions

nothing about fonts.

`g_font_title`, `g_font_body` and `g_font_small` are built once at startup and retained.

7. What to understand

Nine things about this player are not obvious from listening to it.

1. Nothing ever sleeps

The whole design follows from refusing `sleep_until`. The tune becomes a list of "at sample *N*, do this", and the audio callback — the only thread with a reliable clock — applies each event at its offset inside the buffer it is already filling.

Measured worst error: **1 sample, 0.021 ms**, and the source says that is probably the measuring apparatus rather than the scheduler. The design it replaces is out by 1–2 ms idle and tens under load, *varying*, which is what the ear hears as looseness.

2. Time is an integer until the last possible moment

480 ticks to the quarter note, chosen because it divides exactly by everything ABC can ask for — 1/64 notes, triplets, dots, quintuplets. Then **one** conversion:

```
return (tick * sample_rate * 60 + denom // 2) // denom
```

Computed from the event's absolute tick, not from the previous event's sample, so there is nothing to accumulate. Floating-point timestamps would drift by a fraction of a tick per note — inaudible for a hundred notes and then visibly wrong.

3. The MIDI file is the proof, not a feature

A MIDI file is a public format with other readers: if the tune opens in a notation program with the right pitches, the right lengths and the bar lines in the right places, then the parser and the tick arithmetic are correct, and no amount of listening to a synthesiser could establish the same thing.

Listening tests your ears. A file read by software that was not written here tests your model — and displays it as notation, so a bar that does not add up is visible instead of audible.

4. An accidental holds to the end of the bar

Not to the end of the note. `^F A F` has **two** sharp Fs, and the F in the next bar is natural again. A parser that misses this plays wrong notes in roughly every tune containing an accidental, which is most of them.

Held in one `Int` as seven nibbles, biased by 8 so that a natural sign — an alteration of zero, and a real instruction — is distinguishable from "nothing set".

5. A key signature is arithmetic

Sharps arrive F C G D A E B; flats take that list backwards. Two sharps means F and C are raised, and that is D major. `key_alter` derives it; there is no table to half-fill — which is the shape that let the earlier version's key handling go unnoticed.

Modes are a subtraction on the circle of fifths — Dorian -2, Mixolydian -1, minor -3 — and they matter, because `K:Ador` is in G major's signature and reading it as A major is three accidentals wrong for the whole tune. Folk material uses modes constantly.

6. First and second endings are the point of a repeat

`| : A | 1 B : | 2 C |` plays **A B A C**. Text-duplicating expanders produce A B A B, so a repeated strain ends on the wrong phrase every time.

The mechanism is one rule — at `: |`, replay from the repeat start but stop where the first ending began — and there is no special case for `| 2` at all; the second pass simply falls through into it.

Expanding over **events** rather than text is what makes repeats across line breaks, and inside one voice of several, need no code.

7. The audio thread's contract shapes three unrelated decisions

May not allocate, may not lock, may not raise. Consequences:

- the schedule is `calloc` 'd into a flat block before the unit starts, and only read after
- `midi_to_hz` clamps, because its octave loop would **hang** on a nonsense note — *"a wrong pitch is a wrong note; a hang is silence for the session"*
- `render` is an `fn`, which in this dialect is both non-raising and C-callable, so it goes straight into `AudioUnitSetProperty` with no shim

And the deliberate absence of a lock, which is a judgement rather than an oversight: a torn read costs one frame of one oscillator; a lock can make the audio thread wait on the UI thread, and a late buffer is an audible click.

8. A chord must not advance the clock

When repeats are expanded, every event is re-stamped — and only things that occupy time advance the clock. Get it wrong and `[CEG]` becomes C, E, G one beat apart.

The comment records the symptom because the symptom is misleading: an arpeggio where a chord should be looks exactly like a chord-parsing bug, and the parser is fine. The fault is three stages downstream in the re-timing.

9. The compiler notes in the README are now history

The README lists four compiler behaviours that cost real time during the port. **Three of them no longer reproduce.** From `CLAUDE.md`, retested August 2026 against the shipped compiler:

trap	status
let binds by reference	still true — it is the language, and there is now a warning for the tracked case
+= through a List subscript	fixed; reverting the workaround gives a byte-identical MIDI file
passing mut Struct on to a second function	fixed
a fn returning a heap-owning type	now a clean diagnostic asking for <code>.copy()</code>
reading self while appending to its own List	fixed

The workarounds are still in the source — the chord emitter, the bar mirror and the broken-rhythm handler are written inline because passing `mut Struct` used to crash — and the annotations are still there beside them.

Do not write new code around any of these except the first, and do not cite them as current behaviour. The rule that produced this list is worth keeping: write the four-line reproducer and run it before working around a compiler.

A short list of things that will bite

If you change...	...this happens
ticks to a float	voices drift apart after a few hundred notes
tick_to_sample to accumulate from the previous event	the same drift, faster
the accidental map to store raw alterations	=F silently does nothing
repeat expansion to work on text	endings break; line breaks and voices need special cases
a chord member to take the current tick	[CEG] arpeggiates, one member per beat
the step sort's tie-break	a note is dropped whenever one ends as another begins
a List on the audio thread	an allocation under a 10.7 ms deadline
midi_to_hz's clamp	a nonsense note hangs the audio thread — silence, no crash
a lock around the registers	the audio thread waits on the UI thread; clicks

fonts built inside drawRect:	nil after a while, and a trap inside AppKit
------------------------------	---

Running it

```
cocoamojo --build examples/abcplayer/main.mojo -o /tmp/abcplayer
```

/tmp/abcplayer tunes/carolan.abc	chip voices
... --midi	Apple's DLS synthesiser
... --write=out.mid	a Standard MIDI File, no playing
ABC_SHOT=<path>	draw one frame to a PNG and exit

The synthesiser it plays through is covered in [its own walkthrough](#).